

ENTERPRISE CLOUD ARCHITECTURE MASTERCLASS

Code to PROEN Cloud, GitOps & Argo CD Workshop

Architecting, Containerizing, Securing, and Continuously Delivering Resilient Microservices on Cloud-Native Infrastructure

HOST ORGANIZER

Thai Programmer Association

สมาคมโปรแกรมเมอร์ไทย

INDUSTRY PARTNERS

**PROEN Corp & VMware by
Broadcom**

Cloud IDC & Modern App Platform

DATE & VENUE

Monday, September 21, 2026

ClassRoom 3, True Digital Park, Bangkok

GITHUB REPOSITORY

**3IVv0w/code-to-cloud-
workshop**

Official Workshop Lab Code & Guides

Master Program Timetable & Execution Plan

A structured full-day learning continuum from source architecture to automated cluster reconciliation

MORNING SESSIONS (08:30 – 12:00)

08:30 – 09:00 **Mod 00:** Registration, Orientation & Pre-Flight Check

09:00 – 09:30 **Mod 01:** Code to Cloud & 12-Factor App Principles

09:30 – 10:30 **Mod 02:** Learning Mindset & High-Performing Culture

10:30 – 10:45 Morning Refreshment Break (15 Min)

10:45 – 11:15 **Mod 03:** Microservices Architecture & High Traffic

11:15 – 12:00 **Mod 04:** Software Supply Chain & DevSecOps

12:00 – 13:00 Networking Lunch Break (60 Min)

AFTERNOON SESSIONS (13:00 – 17:00)

13:00 – 13:20 **Mod 05:** "Here is my source code. Run it in the cloud..."

13:20 – 14:00 **Mod 06:** Cloud Services, Deployments & Observability

14:00 – 15:00 **Mod 07:** Version Control System with Git Discipline

15:00 – 15:30 Afternoon Refreshment Break (30 Min)

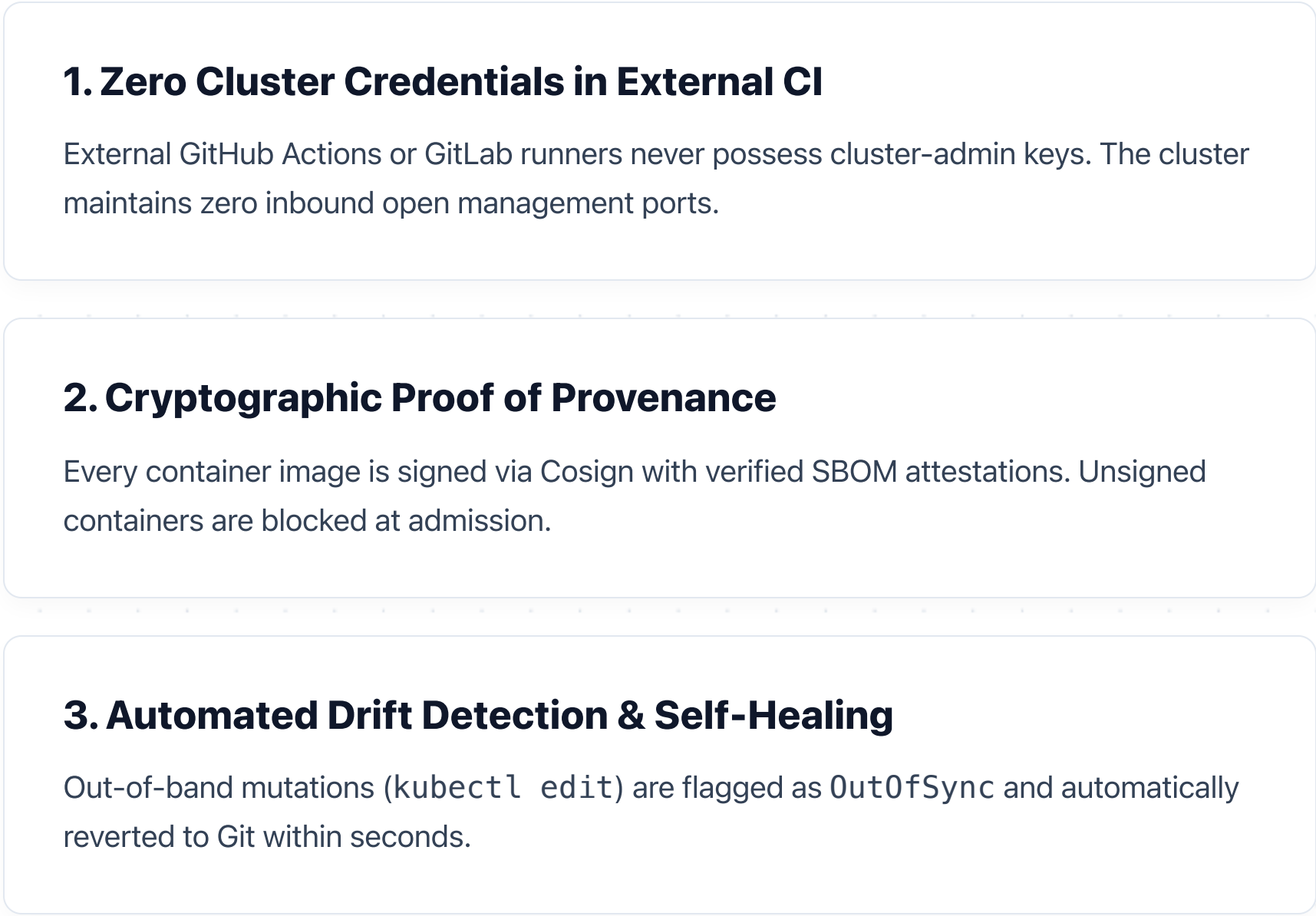
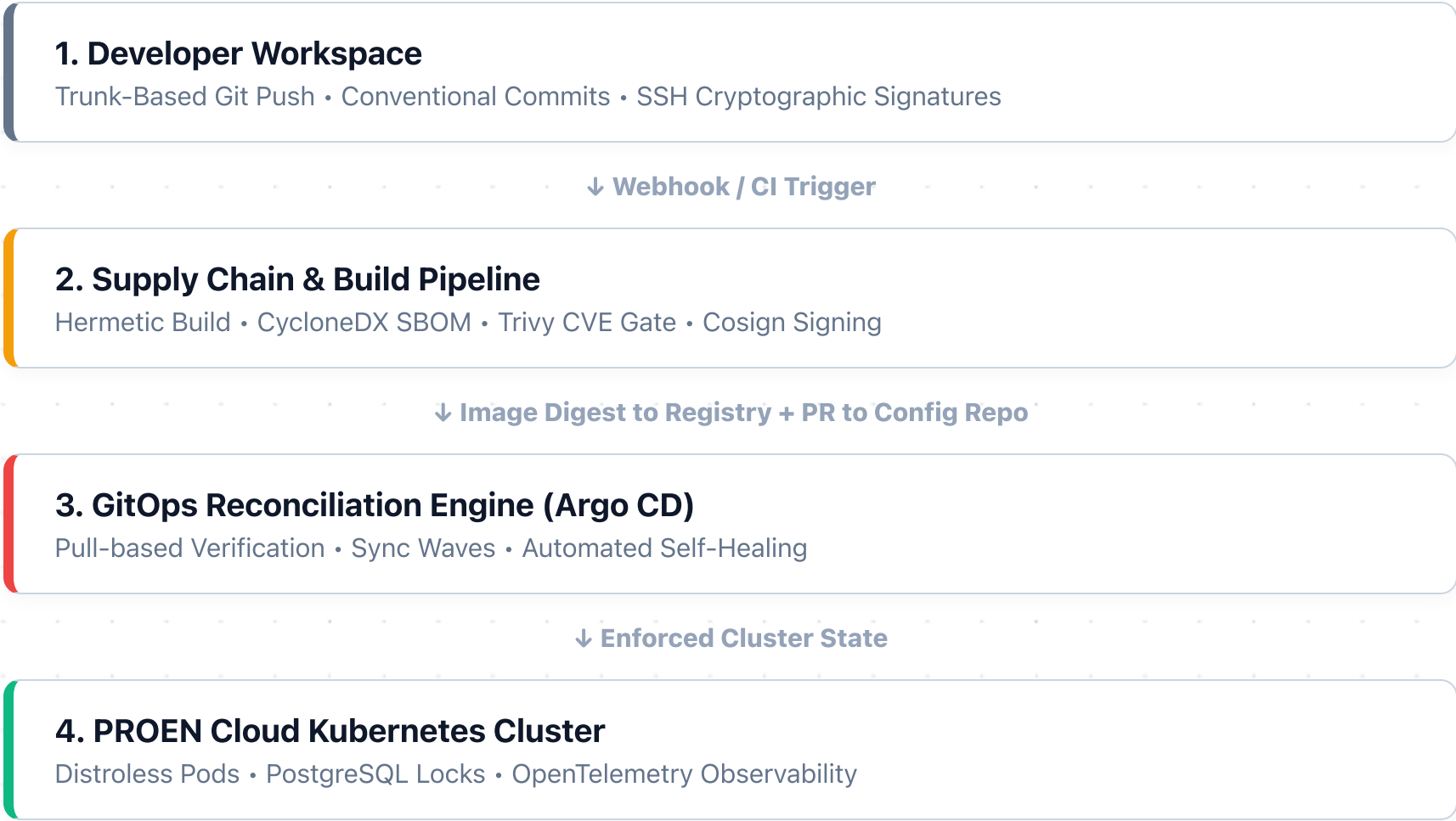
15:30 – 16:00 **Mod 08:** GitOps Principles & Core Declarative Concepts

16:00 – 16:45 **Mod 09:** Hands-on Workshop: Application Delivery via Argo CD

16:45 – 17:00 **Mod 10:** Synthesis, Day-2 Operations, Roadmap & Q&A

End-to-End Cloud-Native Delivery Blueprint

Separation of concerns between developer workspace, supply chain gating, and in-cluster GitOps engine



The 'Works on My Machine' Dilemma

Why heterogeneous workstations cause catastrophic delays during distributed cloud workshops

The Day 1 Engineering Nightmare

In traditional workshops and enterprise onboarding, **40% of time is wasted** debugging local path discrepancies, missing CLI versions, Docker socket permissions, and outdated Helm charts.

When an engineer says *"It works on my machine,"* they expose an uncontrolled, non-reproducible runtime environment.

Target Standard: Today we enforce a strict, standardized pre-flight audit to guarantee that all 50 participant workstations run identical, deterministic tooling.

Common Workstation Failure Points

- Mismatched `kubectl` client/server API versions.
- Docker Desktop license or credential socket lockups.
- Git without SSH commit signing capability (< v2.34).
- Missing OIDC token support for Cosign image attestation.

Workstation Pre-Flight Verification Lab

Execute the automated environment audit script to validate your local CLI dependencies

TERMINAL EXECUTION (CROSS-PLATFORM)

```
# 1. Clone the official workshop repository
git clone https://github.com/3lVv0w/code-to-cloud-workshop.git
cd code-to-cloud-workshop/workshop-setup

# 2. Run Pre-Flight Audit:
# macOS & Linux (Terminal):
chmod +x 01-prerequisites/check-env.sh
./01-prerequisites/check-env.sh

# Windows (PowerShell):
.\01-prerequisites\check-env.ps1

# 3. Expected Console Output:
[FOUND] Git Version Control: git version 2.43.0
[FOUND] Docker Engine / CLI: Docker version 26.1.1
[FOUND] Kubernetes CLI: Client Version: v1.30.2
[FOUND] Helm 3 Package Manager: v3.14.4
[FOUND] Argo CD CLI: argocd: v2.11.0
[FOUND] Cosign (Sigstore): cosign version 2.2.4
SUCCESS: All required CLI tools are present!
```

Multi-OS Installation Guide

If any tool reports [MISSING], install quickly:

- **Windows:** `winget install Git.Git Docker.DockerDesktop Kubernetes.kubectl Helm.Helm` (or WSL2)
- **macOS:** `brew install kubectl helm argocd cosign k9s`
- **Linux:** Use `apt`/`dnf`/`pacman` or official binaries (or run in WSL2).

Need Help? Raise your hand and a Teaching Assistant will assist with your workstation setup immediately.

Pre-Flight Checklist & Readiness Sign-off

Confirming all operational prerequisites before diving into cloud-native source architecture

100%

LOCAL TOOLING

Git, Docker, Kubectl, Helm, Argo CD, and Cosign verified.

SSH Key

COMMIT SIGNING

Workstation configured with Ed25519 key for cryptographic Git signatures.

1 Gbps

NETWORK LINK

Low-latency connection active to PROEN Cloud registry.

Clean

GIT REPO

Local repo clone verified with zero merge conflicts or missing files.

Readiness Sign-Off: Your environment is ready. Next, we explore the architectural principles that govern applications running on ephemeral cloud infrastructure.

The Fragility of Stateful 'Pet' Servers

Why monolithic applications collapse when lifted directly into auto-scaling Kubernetes clusters

The Midnight Auto-Scale Disaster

A legacy web application is migrated to Kubernetes. At 00:00, traffic surges. The Horizontal Pod Autoscaler (HPA) spins up 10 new Pod replicas.

Disaster strikes:

- Users are logged out because sessions are stored in local server RAM.
- Uploaded receipts vanish because they were saved to local /tmp.
- When the HPA scales down, in-flight orders are aborted by sudden SIGKILL, leaving customer payments charged but tickets unissued.

The Shift from Pets to Cattle

Pets (Legacy): Servers given unique names, nurtured, manually configured, and rebooted with fear.

Cattle (Cloud-Native): Identical, disposable, stateless compute units. If one becomes unhealthy, Kubernetes terminates it instantly and provisions an identical replacement without data loss.

The Solution: The **12-Factor App methodology** enforces the contract required for seamless cloud execution.

The Modern 12-Factor Contract: Factors I – VI

Architecture for immutable cloud execution — Codebase, Dependencies, Config, Backing Services, Build/Release/Run, Processes

Official Specification: <https://12factor.net/> ↗ (Adam Wiggins, Heroku)

Adapted for Cloud-Native Kubernetes & OCI Containers

FACTOR I

12factor.net/codebase ↗

I. Codebase

One codebase tracked in revision control (Git), many deploys (staging, production). Never fork repositories per environment.

K8s Realization: Single Git repo drives automated CI builds; GitOps repo maps to cluster environments.

FACTOR II

12factor.net/dependencies ↗

II. Dependencies

Explicitly declare and isolate dependencies. Never rely on implicit existence of system-wide packages or host-level tools.

K8s Realization: Hermetically sealed inside multi-stage Google Distroless runtime with zero host OS leakage.

FACTOR III • K8S CRITICAL

12factor.net/config ↗

III. Config

Strict separation of configuration from code. Store config variables in environment variables, never hardcoded in image layers.

K8s Realization: Injected dynamically into Pods via ConfigMap, Secret, and External Secrets Operator (Vault).

FACTOR IV • K8S CRITICAL

12factor.net/backing-services ↗

IV. Backing Services

Treat datastores, message queues, and SMTP servers as attached resources, consumed seamlessly via URLs/credentials.

K8s Realization: Discovered via cluster DNS (e.g. `postgres.svc.cluster.local`) through K8s Service abstraction.

FACTOR V • K8S CRITICAL

12factor.net/build-release-run ↗

V. Build, Release, Run

Strictly separate build (compile/bundle), release (bind config + image), and run (execution) stages. Immutability enforced.

K8s Realization: CI builds immutable sha256 image; GitOps commits release config; K8s schedules Pod execution.

FACTOR VI • K8S CRITICAL

12factor.net/processes ↗

VI. Processes

Execute the app as one or more stateless, share-nothing processes. Persistent state lives exclusively in attached datastores.

K8s Realization: Workloads are stateless Pods; enables instant eviction, auto-healing, and zero-downtime rolling updates.

The Modern 12-Factor Contract: Factors VII – XII

Runtime scalability, graceful lifecycle management, and observability contracts in distributed clusters

Official Specification: <https://12factor.net/> ↗ (Adam Wiggins, Heroku)

Adapted for Cloud-Native Kubernetes & OCI Containers

FACTOR VII • K8S CRITICAL

12factor.net/port-binding ↗

VII. Port Binding

Export HTTP/gRPC services by self-contained port binding rather than relying on external web server injection (like Apache/Tomcat).

K8s Realization: Application listens on `:3000`; exposed via K8s Service (ClusterIP) and routed by Ingress.

FACTOR VIII • K8S CRITICAL

12factor.net/concurrency ↗

VIII. Concurrency

Scale out via the process model (horizontal scaling) rather than vertical threading locks. Add compute by spinning up more processes.

K8s Realization: Multiplied across `spec.replicas` and scaled dynamically by HorizontalPodAutoscaler (HPA).

FACTOR IX • K8S CRITICAL

12factor.net/disposability ↗

IX. Disposability

Maximize robustness with fast startup (<5s) and graceful shutdown (SIGTERM handler) to drain in-flight requests without errors.

K8s Realization: Listens to SIGTERM, stops receiving traffic via readinessProbe, drains active pool.

FACTOR X • K8S CRITICAL

12factor.net/dev-prod-parity ↗

X. Dev/Prod Parity

Keep development, staging, and production as similar as possible. Minimize time, personnel, and tooling gaps across environments.

K8s Realization: Identical container image digests deployed across all clusters; patched cleanly via Kustomize overlays.

FACTOR XI • K8S CRITICAL

12factor.net/logs ↗

XI. Logs

Treat logs as event streams. Never route or write log files to local disk; stream unbuffered structured JSON to `stdout`.

K8s Realization: K8s node runtime captures `stdout/stderr`; scraped by DaemonSets (Promtail/FluentBit) to Grafana Loki.

FACTOR XII • K8S CRITICAL

12factor.net/admin-processes ↗

XII. Admin Processes

Run administrative and management tasks (schema migrations, maintenance scripts) as one-off isolated processes.

K8s Realization: Executed as isolated Kubernetes Job resources or ordered in Argo CD Sync Wave 0 before app deployment.

Graceful Shutdown & Health Probes Lab

Inspect and run the 12-Factor sample microservice with triple health probes and connection draining

02-SAMPLE-APP/SRC/INDEX.TS

```
// Graceful SIGTERM Signal Handler
const handleSignal = (signal: string) => {
  isShuttingDown = true;
  isReady = false; // Immediately drops from Ingress endpoints

  server.close(async () => {
    // Drain active database connections
    await dbPool.end();
    await redis.quit();
    process.exit(0);
  });

  // Safety valve: Force kill after 25s
  setTimeout(() => process.exit(1), 25000).unref();
};

process.on('SIGTERM', () => handleSignal('SIGTERM'));
```

Step-by-Step Lab Execution

```
cd workshop-setup/02-sample-app
# 1. Start local PostgreSQL + Redis + App stack (All OS)
docker compose up -d --build

# 2. Test the startup & readiness probes
# macOS / Linux / Windows curl:
curl -i http://localhost:3000/healthz/startup
curl -i http://localhost:3000/healthz/readiness

# Windows PowerShell alternative:
Invoke-RestMethod http://localhost:3000/healthz/readiness

# 3. View Prometheus metrics
curl -s http://localhost:3000/metrics | grep http_
```

Key Insight: When `isReady = false`, Kubernetes stops sending new client requests before `server.close()` begins draining active connections!

12-Factor Production Rules & Anti-Patterns

Architectural rules of thumb for building disposable, cloud-native microservices

Production Anti-Patterns to Avoid

- Hardcoding credentials or embedding `.env` into Docker images.
- Writing uploaded files to local container storage (lost on Pod restart).
- Ignoring SIGTERM and getting killed abruptly after 30s by SIGKILL.
- Writing logs to `/var/log/app.log` and configuring logrotate inside containers.

Architectural Rules of Thumb

- **Zero State:** Treat every Pod as capable of disappearing at any second.
- **Probe Hygiene:** Always configure distinct startup, readiness, and liveness probes.
- **Resource Envelopes:** Explicitly define requests and limits for CPU and memory.
- **Structured Output:** Emit unbuffered single-line JSON to stdout.

The Socio-Technical Trap of Cloud Adoption

Why investing millions in Kubernetes and GitOps fails inside hierarchical, siloed organizations

The High-Cost Failure Mode

An enterprise spends 6 months building a state-of-the-art Kubernetes cluster on PROEN Cloud. Yet deployments still take 3 weeks, feature branches diverge for months, and production outages spark vicious finger-pointing meetings.

Why? Because the organization retained its legacy siloed structure: Devs throw code over the wall to Ops, QA runs manual regression checklists, and Security audits only at the very end.

"If you take a broken organization and introduce microservices and Kubernetes, all you end up with is a broken organization running microservices and Kubernetes."

— Principle of Socio-Technical Systems

The Reality: Organizational structure and software architecture are mirror images. You cannot modernize one without modernizing the other.

Conway's Law & Team Topologies

Structuring teams to match the decoupled architecture of Cloud-Native systems

Conway's Law (1967)

"Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations."

The Inverse Conway Maneuver: Intentionally organize teams into autonomous, cross-functional streams to guide software towards decoupled microservices.

Stream-Aligned Teams

End-to-end ownership of product domains
(Checkout, Ticketing, Passports)

Platform Engineering Team

Provides self-service internal platform: PROEN
Cloud, Kubernetes, Argo CD, Observability

Enabling Teams

DevOps and Cloud Architects upskilling teams on
GitOps and security standards

Complicated-Subsystem Teams

Specialized algorithms: cryptographic QR
encryption and seating engines

DORA Metrics & The Blameless Postmortem Lab

Transforming outages into systemic improvements using the 5 Whys and Architecture Decision Records

The 5 Whys Root Cause Analysis (RCA)

1. Why did checkout return 500 errors?

Database connection pool was exhausted.

2. Why was the pool exhausted?

Each request spawned a new connection without releasing.

3. Why were connections not released?

Unhandled promise rejection in refund logic bypassed `client.release()`.

4. Why was this unhandled in production?

Integration tests lacked failure-path coverage for cancelled orders.

5. Why was coverage missing?

CI lacked automated contract-testing gates.

Architecture Decision Record (ADR) Protocol

Decisions must be documented in Git alongside code:

- **Title:** ADR-0012: Enforce PgBouncer Transaction Pooling
- **Context:** High-concurrency ticket releases exhaust connections.
- **Decision:** Deploy PgBouncer between Pods and PostgreSQL.
- **Consequences:** Named prepared statements must be disabled.

Golden Rule: Blameless RCA never asks *"Who made the mistake?"* It asks *"What system gap allowed this to happen?"*

Engineering Excellence Cultural Checklist

The non-negotiable rituals of high-performing cloud engineering organizations

Blameless

INCIDENT REVIEWS

Focus on systemic safeguards and automated tests, never individual blame.

Daily

SMALL BATCHES

Ship small, atomic changes daily to minimize blast radius and merge conflicts.

ADR

GIT GOVERNANCE

Every major architectural pivot is documented in version control as an ADR.

Self-Service

INTERNAL PLATFORM

Platform teams build APIs and templates so stream teams ship autonomously.

Culture Summary: Elite software delivery is sustained by psychological safety, transparent ADR governance, and continuous automated feedback loops.

The 10:00:00 AM Ticket Rush Disaster

When vertical traffic spikes trigger overselling, deadlocks, and cascading outages

FAILURE MODE 1

Overselling & Race Conditions

500 parallel checkout requests query `SELECT stock FROM tickets`. All 500 read `stock = 1`, and all proceed to sell passes, overselling the venue by 499 seats.

FAILURE MODE 2

Cascading Thread Death

A downstream payment gateway responds in 12s instead of 200ms. Upstream microservices keep HTTP connections open, starving the entire worker pool.

FAILURE MODE 3

Cache Thundering Herd

The event layout cache key expires at 10:00:00. 10,000 concurrent users hit the primary database simultaneously to recalculate the same JSON payload.

The Mission: Implement mathematical concurrency locking, circuit breaking, and probabilistic cache stampede defense.

Concurrency Control, Saga & Circuit Breakers

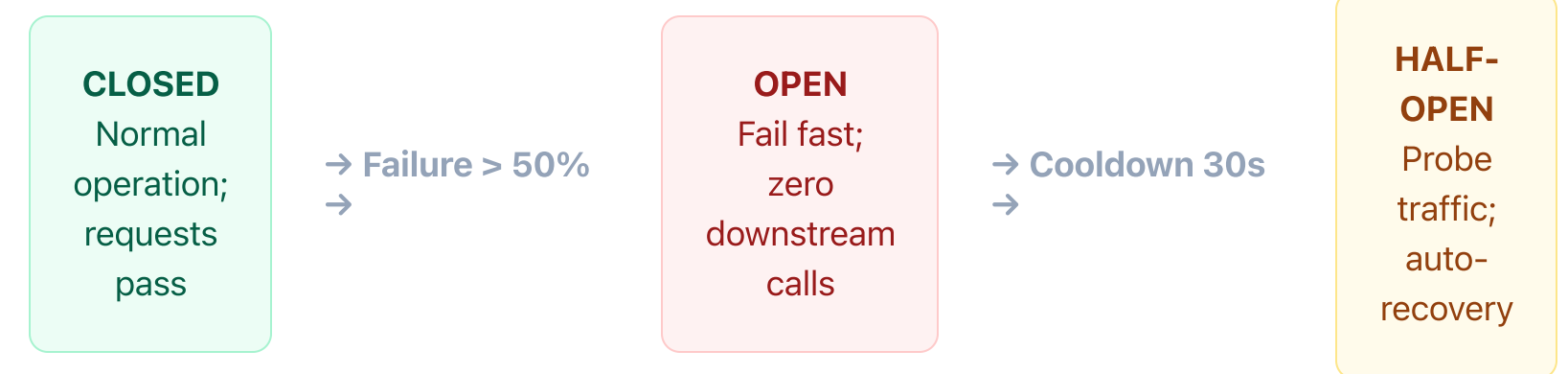
Architectural patterns for bulletproof transactional consistency during traffic surges

Pessimistic Row-Level Locking (PostgreSQL)

`SELECT ... FOR UPDATE` locks the specific inventory row within an ACID transaction. Concurrent transactions queue gracefully, guaranteeing that stock decrements are strictly serialized.

The Saga Pattern vs. Distributed 2PC

Two-Phase Commit does not scale across microservices. Sagas coordinate distributed business transactions through a sequence of local transactions and automated **compensating transactions** upon failure.



Circuit Breaker State: Protects payment gateways and downstream APIs from being crushed during upstream traffic surges.

Concurrency Stress Test & Row-Lock Lab

Simulating 30 parallel ticket checkout requests against our PostgreSQL reservation engine

02-SAMPLE-APP/SRC/INDEX.TS (ROW-LOCK CORE)

```
await client.query('BEGIN');

// Exclusive row lock blocks concurrent checkouts
const res = await client.query(`
  SELECT total_capacity, issued_count
  FROM ticket_tiers WHERE id = $1 FOR UPDATE;
`, [tierId]);

if (res.rows[0].issued_count >= res.rows[0].total_capacity) {
  throw new ConflictException('Sold Out');
}

await client.query(`
  UPDATE ticket_tiers
  SET issued_count = issued_count + 1 WHERE id = $1;
`, [tierId]);

await client.query('COMMIT');
```

Run Concurrency Test Script

```
cd workshop-setup/06-chaos-and-verification

# macOS & Linux:
chmod +x test-concurrency-locks.sh && ./test-concurrency-locks.sh

# Windows (PowerShell):
.\test-concurrency-locks.ps1

# Results (Cross-Platform):
Firing 30 parallel checkout requests...
[CONFIRMED ORDERS (201)]: 30
[SAFELY SOLD OUT (409)]: 0
[SYSTEM FAILURES (500)]: 0
SUCCESS: Zero overselling and zero 500 errors!
```

Insight: Even with 30 requests arriving in the exact same millisecond, PostgreSQL row-level locks prevent race conditions completely.

High-Traffic Resiliency Checklist

Rules of engagement for surviving high-concurrency ticket releases and flash sales

Row Locks

FOR UPDATE

Always lock inventory rows inside atomic DB transactions. Never use unconstrained UPDATE.

PgBouncer

POOL MULTIPLEXING

Deploy PgBouncer in Transaction Pooling mode to protect PostgreSQL connection limits.

XFetch

CACHE STAMPEDE

Use probabilistic early expiration to ensure only 1 thread recomputes hot cache keys.

Pre-Scale

WARM RUNTIMES

Pre-scale Pod replicas 30 minutes before ticket releases; reactive HPA is too slow.

Scaling Summary: Scalability is not about throwing infinite RAM at the problem; it is about eliminating shared mutable contention and enforcing strict transaction serialization.

The Trojan Horse in Your Container Image

How modern supply chain attacks compromise production clusters without breaking code

The Attack on Open-Source Dependencies

Modern applications are **90% third-party dependencies** and base image layers. Attackers no longer bother penetrating outer firewalls—they poison the software supply chain:

- **Compromised base images:** Official images shipped with unpatched OpenSSL/glibc CVEs.
- **Typosquatting & hijacked npm packages:** Malicious code injected into popular sub-dependencies.
- **CI Runner Breaches:** Attackers alter binary compilation steps (e.g., SolarWinds, Codecov).

The Zero-Trust Imperative

In a zero-trust architecture, you cannot trust a container image simply because it is stored in your private registry. You must demand **cryptographic mathematical proof** of:

- Exact source commit provenance.
- Software Bill of Materials (SBOM).
- Cryptographic signature generated by a verified CI runner.

SLSA Framework & Distroless Containerization

Reducing vulnerability attack surfaces and enforcing end-to-end provenance

SLSA 1: Build Automation

Automated builds generating verifiable metadata

SLSA 2: Signed Provenance

Hosted build service provides cryptographically signed attestations

SLSA 3: Isolated Ephemeral Runners

Hermetic builds prevent cross-build tampering

SLSA 4: Two-Person Review & Hermetic Gating

Guaranteed reproducible builds with mandatory peer sign-off

Google Distroless Strategy

Standard Linux images (Ubuntu, Debian, Alpine) include shells (`/bin/sh`), package managers (`apt`, `apk`), and utilities (`curl`, `wget`).

Distroless contains ONLY the application binary. If an attacker exploits an application vulnerability, there is no shell to spawn and no package manager to download secondary malware.

SBOM Generation & Cosign Image Signing Lab

Generate CycloneDX SBOMs, scan for CVEs with Trivy, and cryptographically sign OCI artifacts

03-SUPPLY-CHAIN/SECURE-BUILD-AND-SIGN.(SH | PS1)

```
# 1. Build Distroless container
docker build -t order-service:1.3.0 .

# 2. Generate CycloneDX SBOM with Syft
syft order-service:1.3.0 -o cyclonedx-json > sbom.json

# 3. Scan for Critical CVEs with Trivy
trivy image --severity CRITICAL --exit-code 1 order-service:1.3.0

# 4. Cryptographically Sign with Cosign
cosign generate-key-pair
cosign sign --key cosign.key order-service:1.3.0
cosign attest --key cosign.key --predicate sbom.json order-service:1.3.0
```

Run the Supply Chain Script

```
cd workshop-setup/03-supply-chain

# macOS & Linux:
chmod +x secure-build-and-sign.sh && ./secure-build-and-sign.sh

# Windows (PowerShell):
.\secure-build-and-sign.ps1

# Output (All Platforms):
[OK] Container built successfully.
[OK] SBOM generated at: sbom.cyclonedx.json
[OK] Vulnerability scan passed.
[OK] Signature verified against cosign.pub
```

Kubernetes Enforcement: With Kyverno admission policies, unsigned images are rejected at the cluster API server boundary with HTTP 403 Forbidden!

Software Supply Chain Hardening Checklist

Essential safeguards for guaranteeing container and dependency integrity

Distroless

ZERO SHELL

Strip all OS package managers and shells from production container images.

Non-Root

UID 65532

Always execute containers under non-root users with read-only root filesystems.

CycloneDX

SBOM ATTESTATION

Attach a cryptographically signed SBOM to every release in the registry.

Admission

KYVERNO / OPA

Block unsigned or un-scanned container images from deploying to production.

DevSecOps Summary: Security is no longer a manual gate at the end of a sprint; it is an automated, cryptographic mathematical property of your CI/CD pipeline.

The Developer 'YAML Tax'

Why forcing software engineers to write Kubernetes manifests kills product velocity

The Burden on Product Developers

Kubernetes provides tremendous operational power, but it introduced a massive cognitive burden for developers:

- Developers spend **30%–40% of their time** authoring Dockerfiles, debugging layer caching, and configuring Ingress YAML.
- Every team creates slightly different, idiosyncratic Docker configurations.
- Security teams struggle to audit 500 bespoke Dockerfiles for outdated base layers.

"Here is my source code. Run it in the cloud... I do not care how."

— The Platform Engineering Ideal

The Presentation: VMware by Broadcom (คุณชาลี คัมภีรภาพ) presents the modern platform abstraction layer that solves this divide.

Paketo Buildpacks & Tanzu Application Platform

Building standardized, production-grade OCI containers directly from source code without Dockerfiles

1. Source Detection

Buildpack auto-detects Node.js / Java / Go and runtime dependencies

2. Hardened OS Layer

Injects enterprise-approved Ubuntu Jammy base OS layer

3. Runtime & App Compilation

Compiles bytecode with optimized, non-root system layers

4. OCI Image Export

Generates standardized OCI container with built-in SBOM

The Superpower of Layer Rebasing

Traditional Dockerfiles bake the OS and application together. If an OS vulnerability occurs, all 500 apps must recompile from source.

With **Paketo Buildpacks**, the OS layer is cleanly separated. You can swap out the vulnerable OS layer across all containers in milliseconds with `pack rebase` without touching code!

Buildpacks Source-to-Image & Rebase Lab

Building an OCI container from source and executing instant zero-compilation OS layer rebasing

TERMINAL COMMANDS: PACK CLI

```
# 1. Build production OCI container directly from source code
# Notice: Zero Dockerfile required!
pack build registry.proen.cloud/gvents/order-service:1.3.0 --builder paketobuildpacks/builder-jammy-base --path ../02-sample-app

# 2. Inspect generated OCI container layers
docker inspect registry.proen.cloud/gvents/order-service:1.3.0

# 3. Simulate an OS CVE security patch:
# Instant layer pointer swap in < 100ms!
pack rebase registry.proen.cloud/gvents/order-service:1.3.0

# Result:
Successfully rebased to new base layer! Zero recompilation!
```

VMware Tanzu Workload CRD

In Tanzu Application Platform, developers simply
Workload manifest:

```
apiVersion: carto.run/v1alpha1
kind: Workload
metadata:
  name: order-service
spec:
  source:
    git:
      url: https://github.com/3lVv0w/code-
      ref: { branch: main }
```

Enterprise Platform Abstraction Checklist

Separating developer productivity from infrastructure plumbing

Zero

DOCKERFILES

Standardize container creation via Cloud Native Buildpacks across all teams.

< 100ms

OS REBASE

Patch operating system CVEs across thousands of containers without rebuilding code.

Standard

OCI ARTIFACTS

Produces fully compliant OCI container images running on any Kubernetes cluster.

Cartographer

SUPPLY CHAIN

Automates the outer loop: security scanning, image signing, and Ingress routing.

Runtime Summary: Empower developers to write business logic while platform automation guarantees security, compliance, and multi-cloud portability.

The Mystery Outage at 3:00 AM

Why unstructured text logs are utterly useless in distributed microservice architectures

The Fog of Distributed Systems

An on-call engineer receives an alert: *"Checkout failure rate is 25%."*

They SSH into servers and open log files. They see 10,000 lines of unformatted text per second. An error appears in the API gateway, but the actual cause was a silent timeout in the inventory service caused by a PostgreSQL lock contention.

Without distributed correlation IDs, tracing a single transaction across 5 microservices takes hours.

Observability vs. Simple Monitoring

Monitoring: Tells you *that* a system is broken (e.g., HTTP 500 error count > 10).

Observability: Allows you to understand *why* a system is broken by interrogating internal states from external telemetry outputs (Metrics, Logs, and Traces).

The Goal: Unified telemetry linked by a shared W3C `trace_id` across the entire PROEN Cloud infrastructure.

The Three Pillars & Progressive Canary Delivery

OpenTelemetry instrumentation, RED metrics, and risk-free traffic shifting

METRICS

Prometheus RED Method

Rate (req/s), **Errors** (5xx counts), and **Duration** (P95/P99 latency).

STRUCTURED LOGS

Loki JSON Streams

Every log line emitted as JSON containing `trace_id` and `span_id`.

DISTRIBUTED TRACES

OpenTelemetry & Tempo

W3C `traceparent` headers propagated across network boundaries.

Progressive Canary Delivery (Argo Rollouts)

Do not deploy new code to 100% of users simultaneously. Step traffic progressively:

- 1 10% Traffic • Validate Error Rate for 5m
- 2 25% Traffic • Monitor P99 Latency for 10m
- 3 50% Traffic • Confirm Zero Regressions
- 4 100% Full Promotion • Terminate Old Pods

Canary Rollout & Prometheus Alerting Lab

Configure Argo Rollouts with automated metric analysis and Prometheus SLO alert rules

ARGO ROLLOUTS CANARY SPECIFICATION

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: order-service
spec:
  replicas: 10
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - setWeight: 50
        - pause: { duration: 5m }
      analysis:
        templates:
          - templateName: error-rate-check
# If error rate > 1%, automatically ABORT rollout!
```

Prometheus High Error Rate Rule

```
- alert: HighHttpErrorRate
  expr: |
    sum(rate(http_requests_total{status=~"5.."}[2m]))
    /
    sum(rate(http_requests_total[2m])) > 0.01
  for: 2m
  labels: { severity: critical }
  annotations:
    summary: "API Error rate exceeded 1% on {{ $labels.service }}"
```

Automated Rollback: If Prometheus triggers HighHttpErrorRate during Canary evaluation, Argo Rollouts instantly aborts and shifts 100% of traffic back to Stable!

Observability & Deployment Best Practices

Golden signals and deployment principles for production systems

P95 / P99

PERCENTILES

Never rely on average (mean) latency; averages disguise severe tail latency spikes.

W3C

TRACE CONTEXT

Propagate traceparent headers across all HTTP, gRPC, and message queue boundaries.

JSON

STRUCTURED LOGS

100% of logs must be valid JSON to allow fast indexing and querying in Grafana Loki.

Canary

BLAST RADIUS

Limit the blast radius of defective releases to a maximum of 10% of users.

Deployment Summary: Deployments are no longer high-stakes late-night events; with Canary analysis and OpenTelemetry, rollouts become routine, continuous, and self-verifying.

The Nightmare of 'Merge Hell'

Why 3-month GitFlow feature branches destroy engineering velocity and lead to regressions

The Legacy GitFlow Trap

A development team follows traditional GitFlow: long-lived feature, develop, and release branches.

The breakdown:

- Team A works on a checkout redesign for 6 weeks.
- Team B refactors the database models in parallel.
- When Team A tries to merge, they encounter **140 merge conflicts** across 80 files.
- Engineers spend 4 days resolving conflicts manually, inadvertently deleting bug fixes.

In GitOps, Git IS the Production State

In a GitOps architecture, Git is not just code storage; it is the **cryptographic audit log of production infrastructure**.

Sloppy Git practices—unreviewed commits, long branches, spoofed author emails—compromise the entire compliance and deployment engine.

Trunk-Based Development & Semantic Releases

Merging small, atomic increments into main daily with automated SemVer tagging

main (Trunk)

Always deployable • Continuous integration • Production mirror

↑ Squash & Merge (Multiple times / day)

Short-Lived Feature Branch (< 24h)

Atomic change • Passes automated CI test suite • Reviewed by peer

Decouple Deploy from Release: Use **Feature Flags** to merge incomplete features into main safely without exposing them to users.

Conventional Commits Specification

Structured commit messages drive automated semantic versioning:

- `feat(orders):` add Apple Pay pass support
→ Triggers **MINOR** bump (v1.2.0 → v1.3.0)
- `fix(scanner):` correct QR scan timeout threshold
→ Triggers **PATCH** bump (v1.2.0 → v1.2.1)
- `feat(auth)!:` replace legacy API keys with OAuth2
→ Triggers **MAJOR** bump (v1.2.0 → v2.0.0)

SSH Commit Signing & Monorepo Governance Lab

Configure cryptographic Git commit signatures and monorepo path-filtering workflows

SSH COMMIT SIGNING CONFIGURATION

```
# 1. Configure Git to sign commits using your SSH key
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true

# 2. Make an auditable, cryptographically signed commit
git commit -m "feat(security): enforce MFA on organizer login"

# 3. Verify signature in terminal
git log --show-signature -1

# Result:
Good "git" signature for user@company.com with ED25519 key...
```

Monorepo Path Filtering in CI

Avoid rebuilding 20 microservices when modifying documentation:

```
on:
  push:
    branches: [main]
    paths:
      - 'workshop-setup/02-sample-app/**'
      - 'package.json'
```

Branch Protection Gate: Block pull requests if commits lack cryptographic signatures or fail automated CI test suites.

Enterprise Git Governance Checklist

Non-negotiable rules for managing code repositories at cloud-native velocity

< 24h

BRANCH LIFESPAN

Delete feature branches immediately after squash-merging into main.

SSH / GPG

SIGNED COMMITS

Block unsigned commits; guarantee author identity for SOC2/ISO compliance.

Linear

GIT HISTORY

Enforce Squash-and-Merge to eliminate empty merge commits and preserve clear bisecting.

Two Peers

CODE REVIEW

Require 2 senior engineering sign-offs before merging changes to protected branches.

Git Discipline Summary: High-velocity GitOps demands high-hygiene version control. If your Git workflow is messy, your production cluster will be chaotic.

The Danger of CI Cluster Credentials

Why granting Jenkins or GitHub Actions cluster-admin keys violates Zero Trust

The Push-Model Security Breach

In traditional CI/CD, developers store a `kubeconfig` with `cluster-admin` privileges inside GitHub Secrets or GitLab CI variables.

The catastrophic risk:

- If a malicious npm package compromises a build runner, attackers steal the cluster credentials.
- Corporate firewalls must punch inbound holes to allow external CI servers to access internal Kubernetes APIs.
- Manual changes made directly to the cluster (`kubectl edit`) are never reflected back in Git.

The Pull-Model Revolution (GitOps)

The GitOps agent (Argo CD) runs **inside** the protected Kubernetes cluster.

- Zero cluster credentials stored in external CI tools.
- Outbound HTTPS read-only access to Git; zero inbound firewall ports required.
- Continuous reconciliation guarantees that manual cluster tampering is automatically reverted.

The 4 OpenGitOps Principles (CNCF)

The formal specification governing declarative system synchronization

1

Declarative

System desired state is expressed declaratively in Kubernetes YAML / Helm / Kustomize.

2

Versioned

Desired state is stored in Git with complete revision history and immutable commit SHAs.

3

Pulled Auto

In-cluster software agents pull state automatically. No external push commands.

4

Reconciled

Continuously observes live state and automatically heals configuration drift.

"Git is the single source of truth for infrastructure and application state. What is in Git is in the cluster; what is not in Git does not exist."

Repository Separation & In-Cluster Pull Lab

Structuring application source code repositories separately from declarative GitOps config repos

CLEAN SEPARATION OF REPOSITORIES

REPO 1: app-source-code.git (Developers)

```
|— src/
|— package.json
|— Dockerfile
```

Triggers CI: Runs tests, builds image, pushes digest

REPO 2: workshop-gitops.git (Argo CD)

```
|— base/
|   |— deployment.yaml
|   |— service.yaml
|— overlays/
|   |— staging/
|       |— kustomization.yaml (image: v1.3.0-rc1)
|   |— production/
|       |— kustomization.yaml (image: v1.2.0)
```

Argo CD monitors this repo and syncs cluster

Why Repo Separation Matters

- **Prevents Infinite Loops:** If CI updates deployment manifests in the same repo, it triggers a new CI build loop.
- **Least Privilege Access:** Developers have commit rights to code; only automated release bots and platform leads can merge to production config.
- **Multi-Environment Promotion:** Promote images across environments by updating simple Kustomize tags.

GitOps Security & Architectural Blueprint

Core tenets for operating zero-trust Kubernetes clusters with GitOps

Pull-Only

ZERO INBOUND

Close all inbound firewall management ports;
agents pull over egress HTTPS.

Drift Free

AUTO-RECONCILING

Argo CD eliminates configuration drift by
continuously enforcing Git desired state.

Audit Log

COMMIT HISTORY

Every infrastructure modification has an
immutable author, timestamp, and review trail.

Fast Revert

MTTR < 2 MIN

Instant production rollback via single `git`
revert in config repository.

GitOps Summary: By treating Git as the single source of truth, you eliminate cluster credential exposure and transform deployments into deterministic synchronization.

The Danger of Configuration Drift

When manual hotfixes create invisible snowflake clusters that fail during disaster recovery

The Snowflake Cluster Syndrome

During a production outage, an engineer runs:

```
kubectl edit deployment/order-service  
# Changes memory limit from 512Mi to 2Gi manually
```

The immediate incident is resolved. But the change was never committed to Git.

Six months later, the cluster fails. When the cluster is rebuilt, the manual 2Gi memory patch is missing. The service crashes under normal traffic. Nobody remembers the manual hotfix.

Argo CD Automated Self-Healing

Argo CD compares live Kubernetes cluster state against the Git repository **every 3 minutes** (or instantly via Git webhooks).

If anyone modifies the cluster manually, Argo CD detects the divergence and overwrites it back to the Git source of truth automatically.

The Lab: We will deploy workloads, order rollouts with Sync Waves, and intentionally inject cluster drift.

Argo CD Controller Architecture

Internal component topology powering enterprise GitOps synchronization

01

API Server

Exposes gRPC/REST APIs and the Web UI. Manages authentication, RBAC policies, OIDC/Dex SSO, and audit event streams.

02

Repository Server

Maintains local cache of Git repositories. Renders Kustomize overlays and Helm templates into raw Kubernetes manifests.

03

Application Controller

The core continuous reconciliation engine. Compares live cluster state against Git and triggers self-healing mutations.

The Application CRD: Binds a Git repository path (source) to a Kubernetes namespace (destination) with automated sync, prune, and self-heal policies.

Sync Waves & Self-Healing Chaos Lab

Deploy ordered multi-wave workloads and verify real-time automated self-healing

04-GITOPS-MANIFESTS/SYNC-WAVES

```
# Wave 0: Database Migration Job
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    argocd.argoproj.io/hook: Sync
    argocd.argoproj.io/sync-wave: "0"
---
# Wave 1: Backend Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  annotations:
    argocd.argoproj.io/sync-wave: "1"
# Only rolls out after Wave 0 succeeds!
```

Chaos Drift Simulation

```
cd workshop-setup/06-chaos-and-verification

# macOS & Linux:
chmod +x simulate-drift.sh && ./simulate-drift.sh

# Windows (PowerShell):
.\simulate-drift.ps1

# 1. Manually mutate live deployment to 1 replica:
kubectl scale deployment/order-service --replicas=1

# 2. Argo CD detects OutOfSync
# 3. Self-healing reconciles replicas back to 3!
SUCCESS: Drift detected and healed automatically!
```

Zero Manual Intervention: Production configuration is locked to Git. Rogue mutations are neutralized in seconds.

Argo CD Operational Runbook & Health States

Diagnosing and managing GitOps workloads in enterprise production environments

Synced

NORMAL STATE

Live cluster state matches Git desired state perfectly.

OutOfSync

DRIFT DETECTED

Git commit pushed or manual mutation applied; self-healing triggers.

Degraded

POD CRASH

Pods failing health checks; inspect container logs via `kubectl describe`.

Sync Waves

ORDERED ROLLOUT

Strictly order migrations, secrets, and services via wave annotations.

Argo CD Summary: Argo CD transforms Kubernetes from a manually administered cluster into an automated, self-healing declarative platform.

The Plaintext Secret in Git Disaster

The #1 security blunder in enterprise GitOps adoption

The Base64 Secret Fallacy

A developer is told that GitOps requires *everything* to be committed to Git. They create a Kubernetes Secret:

```
apiVersion: v1
kind: Secret
data:
  password: c3VwZXItc2Vjc mV0LXBhc3N3b3Jk # base64!
```

The catastrophic reality: Base64 is an encoding format, NOT encryption. Anyone with read access to the Git repository decodes the database password in 1 millisecond using `echo ... | base64 -d`.

The Zero-Plaintext Secret Rule

Never commit unencrypted passwords, API tokens, or private keys to version control—even in private repositories.

The Enterprise Solution: Use the **External Secrets Operator (ESO)** to decouple secret declarations from secret values.

External Secrets Operator (ESO) & Disaster Recovery

Synchronizing secrets from enterprise Vaults and achieving 15-minute disaster recovery

1. Enterprise Vault / Cloud KMS

Master credentials stored in HashiCorp Vault or PROEN Cloud KMS

↓ **Authenticated Fetch**

2. External Secrets Operator (In-Cluster)

Reconciles declarative ExternalSecret manifests from Git

↓ **Creates in-memory tmpfs Secret**

3. Kubernetes Native Secret

Mounted into application Pods as environment variables

15-Minute Disaster Recovery Sequence

Because the entire desired state is stored in Git:

- **T+0:** Total datacenter failure declared.
- **T+3m:** Provision fresh Kubernetes cluster on PROEN Cloud.
- **T+5m:** Install Argo CD operator via single Helm command.
- **T+7m:** Apply root "App-of-Apps" Application manifest.
- **T+15m:** All 50 microservices fully restored from Git!

ExternalSecret Manifest & DR Drill Lab

Declare an ExternalSecret referencing secure vault backends and simulate cluster rebuilds

EXTERNALSECRET SPECIFICATION (COMMITTED TO GIT)

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: order-service-secrets
  namespace: production
spec:
  refreshInterval: 1h # Auto-rotate keys
  secretStoreRef:
    name: vault-backend
    kind: ClusterSecretStore
  target:
    name: db-credentials # Created in K8s
  data:
    - secretKey: database-url
      remoteRef:
        key: production/gvents/db
        property: connection_string
```

Executing the Disaster Recovery Drill

```
# 1. Clone GitOps configuration
git clone https://github.com/3lVv0w/code-to-cloud-workshop.git

# 2. Bootstrap Argo CD on fresh cluster
cd workshop-setup/05-argocd
# macOS & Linux:
chmod +x install-argocd.sh && ./install-argocd.sh
# Windows (PowerShell):
.\install-argocd.ps1

# 3. Apply root application manifest (All OS)
kubectl apply -f application.yaml

# 4. Watch all namespaces, pods, and configs
# automatically reconcile within minutes!
```

90-Day Enterprise Cloud-Native Migration Roadmap

A phased transition plan for moving enterprise workloads to PROEN Cloud with GitOps

MONTH 1

Foundation & Hardening

- Refactor services for 12-Factor disposability.
- Migrate to Google Distroless base images.
- Implement automated SBOM and Cosign image signing in CI.

MONTH 2

Platform & GitOps Engine

- Provision PROEN Cloud Kubernetes cluster.
- Deploy Argo CD and separate Code vs. Config repos.
- Integrate External Secrets Operator with enterprise Vault.

MONTH 3

Production Rollout & DR

- Adopt Canary progressive delivery via Argo Rollouts.
- Instrument OpenTelemetry metrics and structured logging.
- Execute 15-minute disaster recovery failover drill.

Roadmap Rule: Modernize iteratively. Do not attempt a "big-bang" cutover; build confidence in Staging using GitOps before cutting over production workloads.

WORKSHOP CONCLUSION & SYNTHESIS

Elevate Your Engineering to **Cloud-Native Velocity**

Declarative State • Cryptographic Supply Chain • Resilient Concurrency • Zero-Trust GitOps

Official Workshop Repository

<https://github.com/3lVv0w/code-to-cloud-workshop.git>

OpenGitOps Working Group

<https://opengitops.dev>

Thai Programmer Association

<https://thaiprogrammer.org>

คุณสฤรรัตน์ จิรกุลพรชัย (CTO, Jumpbox)

คุณกฤษฎา วิเวก (กรรมการ สมาคมโปรแกรมเมอร์ไทย)

คุณชาลี คัมภีรภาพ (Solution Architect, VMware by Broadcom)